Recitation Assignment #3

October 9, 2024

Today's recitation will involve the general form of gravity, and, in particular, you will show using a computer that two bodies will orbit one another in an elliptical orbit. Moreover, you will use numerical methods to determine the optimal speed for a particle traveling in a circular orbit. Note, for simplicity, when I use the symbol $\odot$, it means "of the sun", and when I use $\oplus$, it means, "of the earth."

You may complete this in class. However, if you are unable to do so, it is expected that you complete this for recitation next time. Remember to submit your program to Kendall and Professor Cruz via email when you are done.

If you have any questions, please ask.

New Concepts:

Physical:

- Central Gravity
- Two body interaction

Computational:

- Mag and Pow functions

We will model the earth-sun system. Since the sun is much more massive than the earth, we will be able to make the assumption that the sun is stationary.

1. Open up your laptop and begin a new Jupyter Notebook

2. Make sure you always include the line "`from vpython import *`". Remember, this is how your script accesses the vpython functions you have been using.

3. Create two spheres. Make one yellow, and at the origin, and the other should look like the earth, and at a position 1.5×10^{11}m (1 "Astronomical Unit" or AU) from the sun, and along the x-axis:

One of the most efficient ways to program is to define all of your physical and other parameters first at the top of the program. A partial list might look like:

```
AU=1.5e11 #Astronomical unit in meters

G=6.67e-11 #Newton's constant in N m^2/kg^2

Msun=2e30 #Mass of the sun in kg

Mearth=6e24 #Mass of the earth in kg

Yr=3.15e7 #Length of the year in seconds
```

and so forth. These parameters are all in MKS (meters, kilometers, seconds) units. Python is not keeping track of the units, so you have to do it yourself.

You may choose to add other parameters. Thus, when you create the earth, it might look like:

```
Earth=sphere(pos=vec(1*AU,0,0), radius=0.1*AU)
```

The radius, obviously, is gargantuan compared to the real earth, but this allows it to be seen. As a reminder, "vec" is a shorthand used by vpython for vectors.

4. Add the following lines to your code:

```
scene.range=1.3*AU

scene.autoscale=0
```

The first line makes the viewscreen large enough to see a bit beyond the "orbit" of the earth, and the second command tells the code not to automatically resize the picture if the earth flies out of the field of view.

5. Here is where we start to add physics.

The sun has a mass of 2×10^{30} kg, and the earth has a mass of 6×10^{24} kg. Set both of these. For example, the mass of the sun can be set by writing: sun.m=Msun and similarly for the earth.

Give the earth an initial velocity of 35 km/s =35000 m/s in the y-direction. This can be done by entering:

```
Earth.v=vec(0,35000,0)
```

6. In the non-relativistic limit

$$\Delta\vec{r} = \vec{v}\Delta t$$

At each timestep, you have to evolve the position of the earth, according to this formula. Put in the following additions to your code:

```
dt=0.005*yr

t=0


while (t < 2*yr):

    rate(20)

    earth.pos = earth.pos + earth.v*dt

    t=t+dt
```

The first command tells the code what the value of Δt is. Remember, you've already defined a variable 'yr' to represent the number of seconds in a year. The second command tells it to "start the clock" at t=0.

7. As we saw, the earth just flies in a straight line, when, in fact, we know that it moves around the sun. What makes it move? Gravity! The force due to gravity can be expressed as:

$$\vec{F}_{\oplus} = \frac{-GM_{\odot}M_{\oplus}}{r^3{}_{sun-earth}}\left(\vec{r}_{\oplus} - \vec{r}_{\odot}\right)$$

Where $G = 6.67 \times 10^{11}\,\frac{Nm^2}{kg^2}$ is the gravitational constant.

In order to compute the rest of the terms in the force, you may need to do a bit of searching on the vpython or python webpages, or using google.

Incidentally, the reason that the denominator has an r^3 (rather than r^2) is that the difference in vectors in the numerator divided by one power of r produces the unit vector, $\hat{r}$ (think about it).

You'll need to use the "mag" and "pow" functions. E.g:

```
F=G*sun.m*earth.m*(sun.pos-earth.pos)/pow(mag(sun.pos-earth.pos),3)
```

Remember, this needs to be **in the loop**, and it needs to be computed every time. Thus, the velocity of the earth needs to be updated, as well as that of the sun:

$$\Delta\vec{v}_{\oplus} = \frac{\vec{F}}{m_{\oplus}}\Delta t$$

$$\Delta\vec{v}_{\odot} = \frac{-\vec{F}}{m_{\odot}}\Delta t$$

Notice the minus sign. Just defining the variable, F, isn't sufficient. You need to apply it to an equation.

Within the loop, calculate the force, and update the velocity.

Notice, too, that you'll need to update the position of the sun and the earth in terms of their velocities. However, since the sun is far more massive, it won't move very much.

8. Put in a blue "curve" following the position of the earth. (You did this in recitation #2, so you may want to look at your program to see what you did).

9. Adjust the initial velocity of the earth so that it moves more like a circular orbit.

10. Run the code for 20 years rather than 2. After doing so, make sure that the orbit of the earth is a perfectly repeating ellipse (at least to the precision of the program). If it isn't, reduce the size of your dt (you may want to correspondingly increase the argument of the rate() function to make sure your program doesn't run too slowly).

11. Put Jupiter into your system. Find the mass, distance from the sun, and orbital velocity of Jupiter by using google. This is more complicated than it might look. In particular, keep in mind to:

    a. Expand the size of your view window to accommodate the new planet.
    b. Compute the ***total*** force on each of the three bodies. That means that you should compute three combinations of forces and get the total. For example, the force on earth would look like:

$$\vec{F}_{\oplus} = \vec{F}_{\oplus - Jupiter} + \vec{F}_{\oplus - \odot}$$

Be careful with your signs, and don't forget you'll need to calculate three of these net forces: one for earth, one for Jupiter, and one for the sun!

    c. Put a trail on Jupiter

12. Observe the system. What happens if you change the mass of Jupiter? Make it 100 times larger than normal (roughly 10% the mass of the sun).

13. As always, send your solution (your '.ipynb' jupyter file) to Kendall and Professor Cruz via email when you are done.